GoCycle Requirements & Design Report

Gavin Kyle - 30196910

Omar Almishri - 30246524

Ethan Bryant - 30234163

Matthew Hajas - 30213765

1. Requirements classification

Each requirement falls into one of three categories. **Functional** requirements (FR) say what the system must do. **Quality** requirements (QR) say how well it must do it. **Constraints** (C) are fixed boundaries on the solution. Functional requirements are grouped by area of the system, and quality requirements are tagged with the attribute they address. Every row names the class or file that enforces or sets it.

Functional requirements:

Riders and accounts:

ID	The system shall...	Enforced in
FR-1	Let a person sign up as a casual rider, a member or a student, and switch between registered riders.	`GoCycleSystem.signUp*`, menu option 1
FR-2	Give members the member rate only while the membership is active; a lapsed member pays the casual rate.	`Member.isActive()`
FR-3	Give students the student rate only while their verification is current; otherwise the casual rate applies.	`Student.isVerified()`
FR-4	Allow a rider at most one unfinished trip at a time.	`TripService.startTrip`

Fleet and stations:

ID	The system shall...	Enforced in
FR-5	Support three vehicle kinds: pedal bike, e-bike and e-scooter.	`VehicleType`, `VehicleFactory`
FR-6	Show every zone, station and docking point, with the docked vehicle, its battery and whether it can be rented.	`RiderApp.showStations`
FR-7	Rent out only vehicles whose status is AVAILABLE.	`Vehicle.checkRentable()`
FR-8	Refuse to rent an electric vehicle below 20% battery, and tell the rider why.	`ElectricVehicle.checkRentable()`, `Battery`
FR-9	Drain electric vehicles while they are ridden (e-bike 1%/min, e-scooter 2%/min) and charge them while docked (1% per 5 min).	`ElectricBike`, `ElectricScooter`, `ElectricVehicle.rest`

Starting a trip:

ID	The system shall...	Enforced in
FR-10	Accept credit card, transit card and phone wallet payments.	`PaymentType`, `PaymentMethod`
FR-11	Refuse a payment type that the station's zone does not accept, and list the accepted ones.	`Zone.allows`, `TripService.startTrip`
FR-12	Rent only a vehicle that is docked at the station where the rider stands.	`DockingStation.findPointHolding`
FR-13	On start: hold a refundable $25.00 deposit, unlock the point, mark the vehicle IN_USE, record the trip and fix the rider's fare for the whole trip.	`TripService.startTrip`, `DepositHold`
FR-14	Announce the unlock, the point number and the deposit on the station kiosk.	`StationKiosk.announce`

Returning a vehicle:

ID	The system shall...	Enforced in
FR-15	When the chosen return station is full, send the rider to the nearest station with a free point, with its distance, and keep billing.	`TripService.requestReturn`, `StationLocator`
FR-16	Report that the network is full when no station has a free point.	`TripService.requestReturn`

ID	The system shall...	Enforced in
FR-17	End a trip only once the docking point confirms the lock. If the latch fails, keep the trip running and ask the rider to push the vehicle in again.	`DockingPoint.dock`, `TripService.confirmDocked`
FR-18	Once the lock is confirmed: charge the fare, release the deposit, end the vehicle's use and issue a receipt.	`TripService.confirmDocked`, `Trip.complete`
FR-19	If the charge is declined, unlock the vehicle again and keep the trip running.	`TripService.confirmDocked`

Pricing and receipts:

ID	The system shall...	Enforced in
FR-20	Price a trip as unlock fee + per-minute rate × billed minutes. Partial minutes round up, with a 1-minute minimum. Rates: casual $1.25 + $0.35/min, member $1.25 + $0.20/min, student $1.25 + $0.25/min.	`BasePricing` and its subclasses, `Trip.durationMinutes`
FR-21	Add a flat $5.00 surcharge to trips longer than 120 minutes.	`LongTripSurcharge`
FR-22	Deliver each receipt to the app inbox, by email, or both, as the rider chooses; let the rider change that choice.	`ReceiptSender`, `ReceiptDelivery`
FR-23	Show the rider their trip history, the running cost of an active trip and their latest receipt.	`RiderApp.showTrips`

Faults and maintenance:

ID	The system shall...	Enforced in
FR-24	Let a rider report a fault (flat tire, broken brakes, low battery, system fault, other) with an optional note, for the vehicle they ride or a docked one.	`FaultService.reportByRider`
FR-25	Let electric vehicles report their own faults: low battery once per discharge, and occasional system faults while docked.	`ElectricVehicle.reportSelfFault`, `FleetSimulator`
FR-26	Take a vehicle with a repair-type fault out of service right away. A vehicle that is out on a trip goes out of service when it is docked.	`Vehicle.markOutOfService`
FR-27	Notify the maintenance crew of every fault that needs a repair. Return the vehicle to service 45 simulated minutes after the report, once all its faults are fixed.	`MaintenanceCrew`
FR-28	Show a notice to a rider whose vehicle reports a fault by itself mid-trip.	`RiderApp.noticeForCurrentRider`

Control room, zones, language and time:

ID	The system shall...	Enforced in
FR-29	Every 30 minutes, move one ready vehicle from a full station to a station with nothing to rent. If the destination will not lock, leave the vehicle where it was.	`FleetSimulator.rebalance`, `ControlRoomService.rebalance`
FR-30	Publish a fleet event for every trip start, redirect and end, and for every fault, return to service, rebalance and failed receipt.	`FleetEventPublisher`, `FleetEvent.Kind`
FR-31	Give each zone its own accepted payment types and kiosk branding (colours, logo).	`Zone`, `ZoneBranding`
FR-32	Show all rider and kiosk text in English or French, with one system-wide language setting that changes every screen at once.	`LocalizationService`
FR-33	Read menu headings aloud in the current language when the rider turns this on and the computer has a speech engine.	`ReadAloud`, `TextToSpeech`

Quality requirements:

ID	Attribute	Requirement	Enforced in
QR-1	Reliability	A refused trip request leaves no side effects: no hold, no unlocked point, no vehicle left in use. Every check runs before any state changes.	`TripService.startTrip`
QR-2	Reliability	A receipt channel that fails does not undo the completed, charged trip or stop the other channels. The failure is recorded on the receipt and reported to the control room.	`TripService.deliver`
QR-3	Reliability	A vehicle is never left docked nowhere. A rebalance releases the origin point only after the destination point locks.	`ControlRoomService.rebalance`
QR-4	Reliability	The clock thread and the menu thread never interleave domain changes, and one failed clock tick does not stop the clock.	`GoCycleSystem.atomically`, `startClock`
QR-5	Accuracy	Money is rounded to the cent after each calculation and shown as Canadian dollars.	`Money`
QR-6	Usability	Every refusal tells the rider why, in the current language, on the kiosk.	`TripException`, `RentalRefusal`, `StationKiosk`
QR-7	Usability	Menus ask again after invalid input instead of failing, and the app exits cleanly when input closes.	`Console`
QR-8	Usability	A language change applies at once to every screen, receipt and email.	`LocalizationService`, `Receipt.toText`
QR-9	Accessibility	Menus can be read aloud, and setting `NO_COLOR` turns off terminal colour.	`ReadAloud`, `Ansi`
QR-10	Maintainability	A new rider kind, fare rule, payment kind, receipt channel or event subscriber can be added without changing `TripService`.	Interfaces in sections 3 and 4
QR-11	Maintainability	Every message key is defined in both languages, and every enum shown to riders is translated.	`LocalizationTest`
QR-12	Testability	Time, the lock sensor, randomness and speech can be injected, so tests are deterministic.	`SimulationClock`, `BooleanSupplier`, `Random`, `TextToSpeech`
QR-13	Security and privacy	Card numbers are shown masked to the last four digits, the full credit card number is never stored, and the wallet token is never shown.	`CreditCardPayment`, `TransitCardPayment`, `PhoneWalletPayment`
QR-14	Robustness	A speech engine failure is silent, a missing config file means defaults, and a missing translation falls back to English, then to the key itself.	`SystemTextToSpeech`, `GoCycleSystem.create`, `InMemoryLocalizationService`

Constraints:

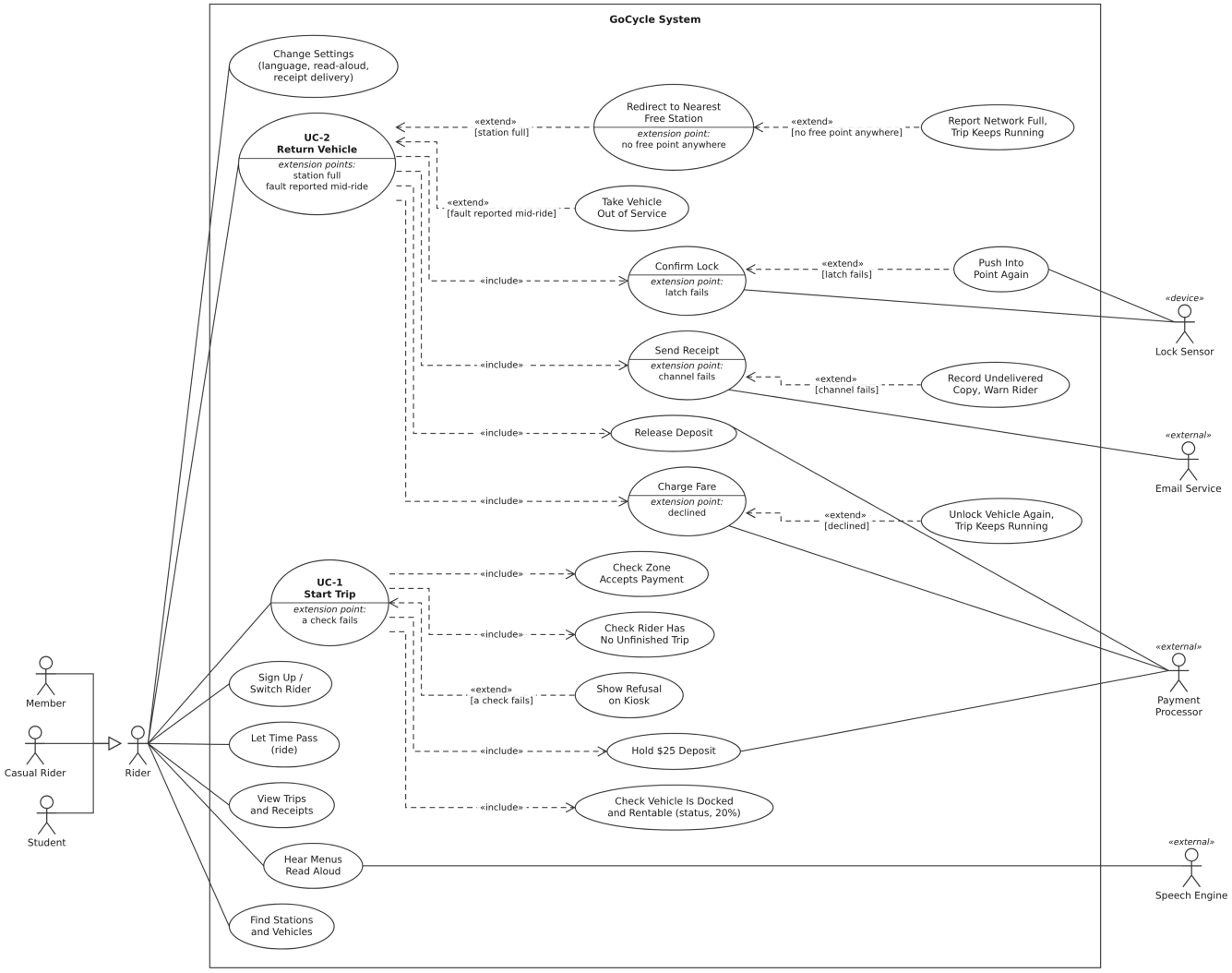
ID	Constraint	Source
C-1	Java 24 or later, built with Maven. JUnit Jupiter 5.11 is the only dependency, and only for tests.	`pom.xml`, README
C-2	A terminal application: the rider app and every station kiosk share one console. There is no GUI or web front end.	`RiderApp`, `StationKiosk`
C-3	All state is in memory and lost on exit. Only emailed receipts persist, as text files in `data/outbox`.	`GoCycleSystem`, `EmailReceiptSender`
C-4	Payment processing and email are simulated: every charge is approved, and an email is a file.	`AbstractPaymentMethod`, `EmailReceiptSender`

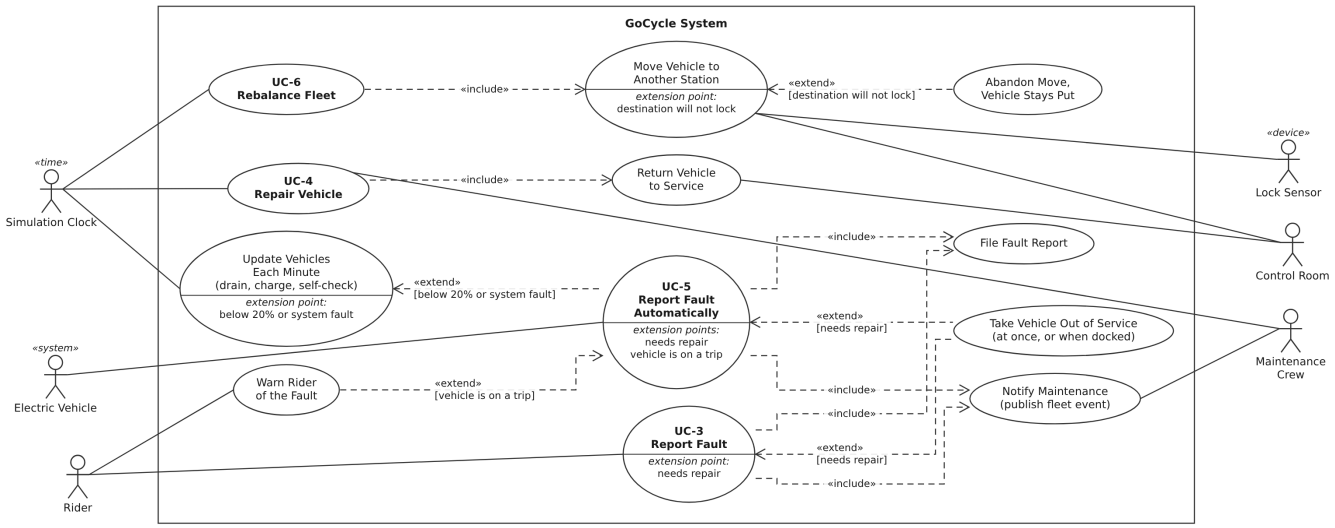
ID	Constraint	Source
C-5	Zone payment rules and branding come from `config/zones.properties`, read once at start-up.	`GoCycleSystem.create`
C-6	Exactly two languages: English and French.	`InMemoryLocalizationService`
C-7	Speech uses only the operating system's own engine (PowerShell System.Speech, `say`, `espeak` or `spd-say`); nothing extra is installed.	`SystemTextToSpeech`
C-8	The launch scripts target Windows.	`start-gocycle.cmd`, `run-rider.cmd`
C-9	Trips, charging and repairs follow a simulated clock (15 simulated seconds per real second by default), not wall time.	`SimulationClock`, `FleetSimulator`
C-10	The seeded network is fixed: 2 Calgary zones, 5 stations, 16 vehicles and 3 demo riders.	`GoCycleSystem.seedNetwork`

2. Use case model

The model has 13 use cases and 12 actors. The **Rider** is the primary actor for trips and is specialized into Casual Rider, Member and Student, who follow the same flows at different fares. An **Electric Vehicle** and the **Simulation Clock** start the automatic use cases. Payment Processor, Email Service, Lock Sensor, Speech Engine, Maintenance Crew and Control Room are secondary actors. UC-1 to UC-6 are the main scenarios and get the most detail below; UC-7 to UC-13 are the simpler supporting use cases.

Use case diagrams:





Use case descriptions:

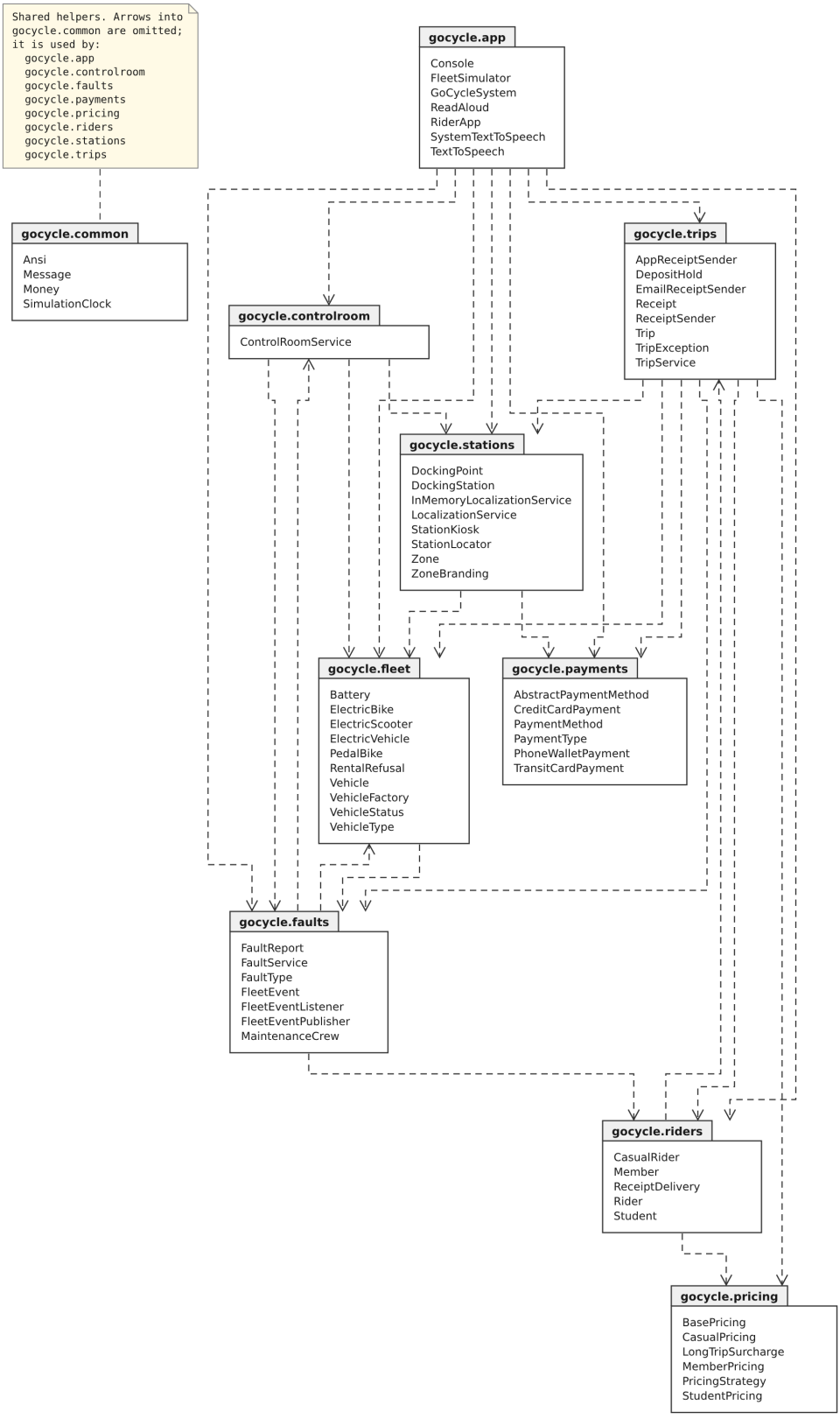
Use case	Actors (primary; secondary)	Trigger and precondition	Main flow	Alternative and failure handling	Outcome
UC-1 Start Trip (FR-4, FR-7, FR-8, FR-10 to FR-14)	Rider (Casual, Member or Student); Payment Processor	Rider picks “Start a trip” (option 3). Rider is signed in and at a station.	Pick station and vehicle → pick payment type (refused types are marked) → system checks: no unfinished trip, zone accepts the payment, vehicle docked here and rentable (AVAILABLE, 20%+ battery if electric) → hold $25.00 deposit → unlock point, vehicle IN_USE, trip recorded with fare fixed, TRIP_STARTED → kiosk shows point and deposit	No vehicles docked: ends. Zone refuses payment: kiosk lists accepted types, rider picks again. Battery under 20%, vehicle out of service or in use, or an unfinished trip: kiosk shows the reason, ends. Back at any step: nothing changes.	Trip ACTIVE, deposit held, vehicle IN_USE. On any failure nothing changes (QR-1).
UC-2 Return Vehicle (FR-15 to FR-22, QR-2)	Rider; Lock Sensor, Payment Processor, Email Service	Rider picks “Return my vehicle” (option 5). Rider has an unfinished trip.	Pick return station → free point found, trip AWAITING_DOCK → push vehicle into the named point → sensor confirms lock → price billed minutes (+$5.00 over 120 min) and charge → release deposit, complete trip, vehicle AVAILABLE → send receipt by app, email or both, TRIP_ENDED → kiosk shows minutes, charge and deposit	Station full: redirected to nearest free station with its distance (TRIP_REDIRECTED); rider rides on at 4 min/km or declines, trip keeps billing. Network full: kiosk says so, trip keeps running. Latch fails or point taken: push into the next free point. Charge declined: vehicle unlocked again, trip keeps running. Fault reported mid-ride: vehicle goes OUT_OF_SERVICE once docked. Receipt channel fails: trip stays completed, failed channel recorded, RECEIPT_FAILED, rider warned.	Trip COMPLETED with end station, time and cost; deposit released; receipt delivered. A trip never ends while the vehicle is unlocked.
UC-3 Report Fault (FR-24, FR-26, FR-27)	Rider; Maintenance Crew (notified)	Rider picks “Report a problem with a vehicle” (option 6). Rider is signed in.	Choose the vehicle being ridden, or a station and a docked vehicle → pick fault type, add optional note → file report F-n → publish FAULT_REPORTED; crew queues a repair due in 45 min → show report ID; kiosk thanks the rider for a docked vehicle	Needs repair: a docked vehicle goes OUT_OF_SERVICE at once; a ridden one is flagged, goes out of service when docked, and the rider is asked to return it. Low battery: informational, no repair queued. No vehicles docked at the station: ends.	Report stored and published. A repair fault keeps the vehicle unrentable until UC-4.

Use case	Actors (primary; secondary)	Trigger and precondition	Main flow	Alternative and failure handling	Outcome
UC-4 Repair Vehicle (FR-27)	Simulation Clock (trigger), Maintenance Crew; Control Room	Every simulated minute. At least one repair job is open.	Crew checks open jobs → resolves each job past its due time whose vehicle is OUT_OF_SERVICE → if the vehicle has no other open job, control room returns it to service, BACK_IN_SERVICE	Vehicle still on a trip: job waits until it is docked. Other open faults: vehicle stays out of service until the last is fixed.	Due reports resolved; a vehicle with no open faults is AVAILABLE again.
UC-5 Report Fault Automatically (FR-25, FR-28)	Electric Vehicle; Maintenance Crew, Rider	Extends UC-13 Update Vehicles Each Minute: charge drops below 20% on a ride, or a docked, rentable EV hits the 0.1% per-minute system-fault chance.	Vehicle files its own report (no rider) → publish FAULT_REPORTED	System fault (needs repair): vehicle out of service, crew queues a repair. Low battery: reported once per discharge, re-armed at 20%, no repair. Vehicle on the signed-in rider's trip: the rider gets an on-screen notice.	Report stored and published. A repair fault keeps the vehicle unrentable until UC-4.
UC-6 Rebalance Fleet (FR-29)	Simulation Clock; Control Room, Lock Sensor	Every 30 simulated minutes. A full station has a ready vehicle, and another station has a free point but nothing to rent.	Vehicle IN_TRANSIT → staff dock it at the destination (up to 10 latch attempts) → origin point released → status restored → REBALANCED	Destination will not lock: vehicle stays at its origin, REBALANCE_FAILED. No suitable pair of stations: nothing happens.	One vehicle moved. A vehicle is never left docked nowhere (QR-3).
UC-7 Sign Up / Switch Rider (FR-1)	Rider	Rider picks option 1.	Pick an existing rider, or enter name and email (plus student number) to register a casual rider, a member ($15.00/month, active 1 month) or a student (verified 8 months)	Blank input is asked again. Back: nothing changes.	Chosen rider is signed in; their fare is shown.
UC-8 Find Stations and Vehicles (FR-6)	Rider	Rider picks option 2.	List each zone with its accepted payments, each station with ready and free counts, and each point with its vehicle, battery and readiness	None: read only.	Network shown in the current language.
UC-9 Let Time Pass (ride) (C-9)	Rider	Rider picks option 4.	Enter 1 to 240 minutes → simulation advances on top of the background clock (15 simulated seconds per real second)	Out-of-range input is asked again.	Batteries, fares and repairs move forward.

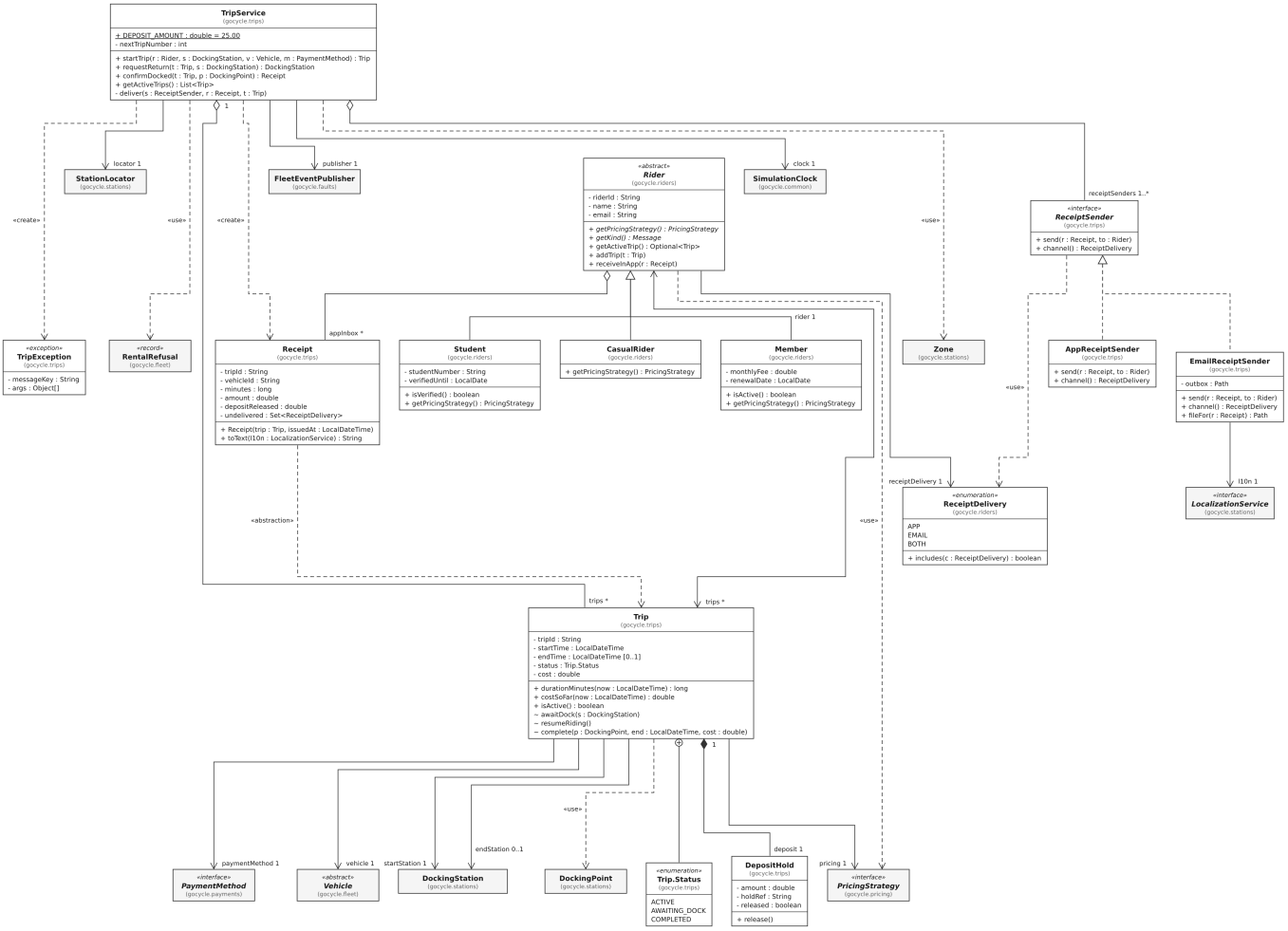
Use case	Actors (primary; secondary)	Trigger and precondition	Main flow	Alternative and failure handling	Outcome
UC-10 View Trips and Receipts (FR-23)	Rider	Rider picks option 7.	Show trip history, the live cost of an active trip, the latest in-app receipt, and the email folder when email receipts are on	None: read only.	Rider sees their trips and receipts.
UC-11 Change Settings (FR-22, FR-32, FR-33)	Rider	Rider picks option 8.	Switch the system language (EN/FR, every screen at once), turn read-aloud on or off, or set receipt delivery (app, email, both)	No speech engine: read-aloud cannot be turned on and the rider is told.	Settings apply at once.
UC-12 Hear Menus Read Aloud (FR-33, QR-9)	Rider; Speech Engine	Read-aloud is on and the computer has a speech engine.	Each menu heading is spoken in the current language	Engine fails: shut down silently, menus carry on.	Menus are spoken as they appear.
UC-13 Update Vehicles Each Minute (FR-9, FR-25)	Simulation Clock	Every simulated minute.	Drain batteries on trips (e-bike 1%/min, e-scooter 2%/min) → charge docked ones (1% per 5 min) → run the 0.1% system-fault check on docked, rentable EVs	Charge below 20% or a system fault: starts UC-5.	Battery levels stay current.

3. Class diagram

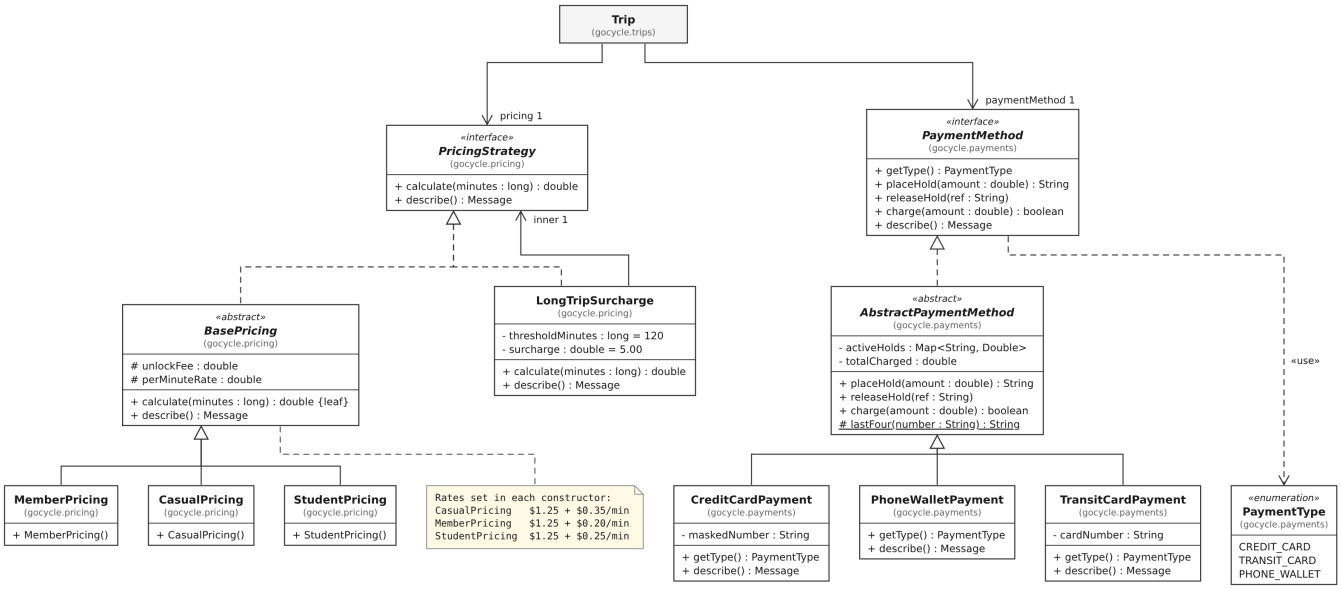
Package overview:



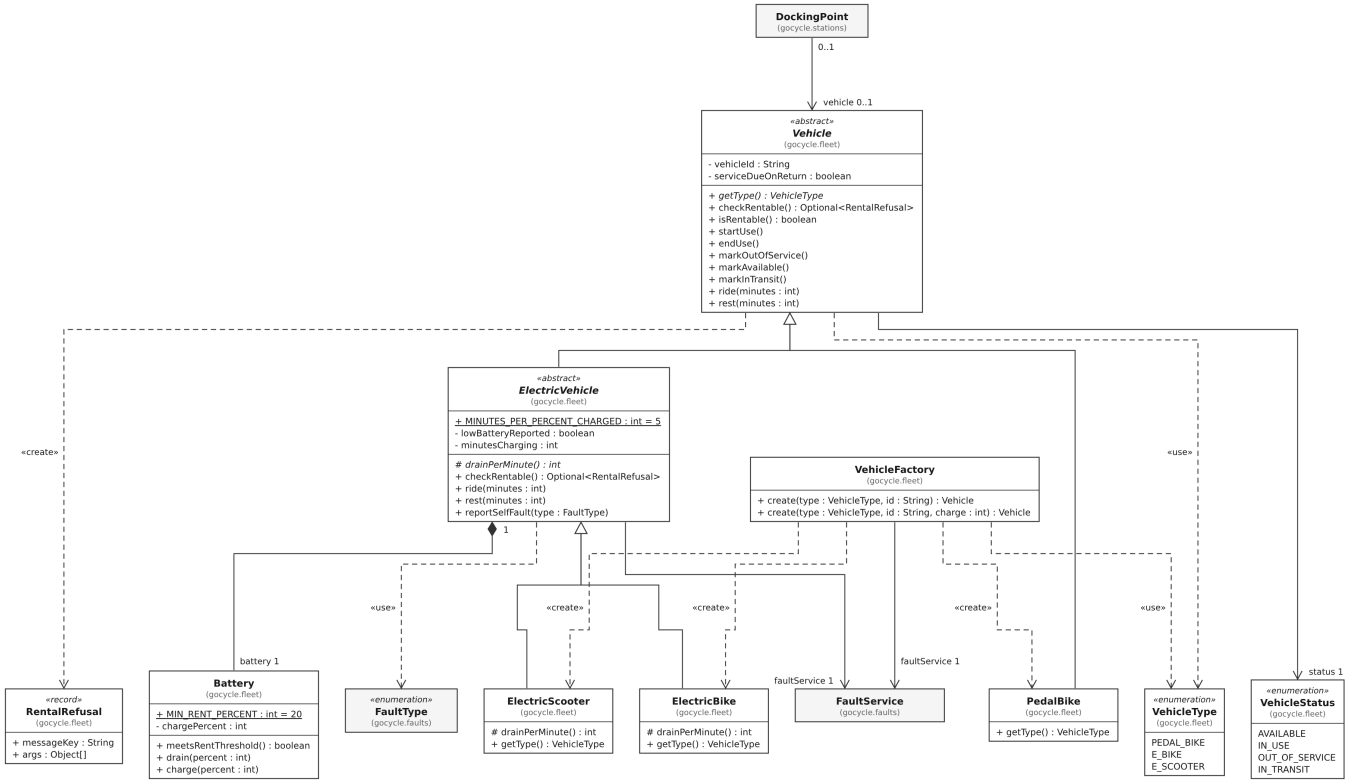
Trips and riders:



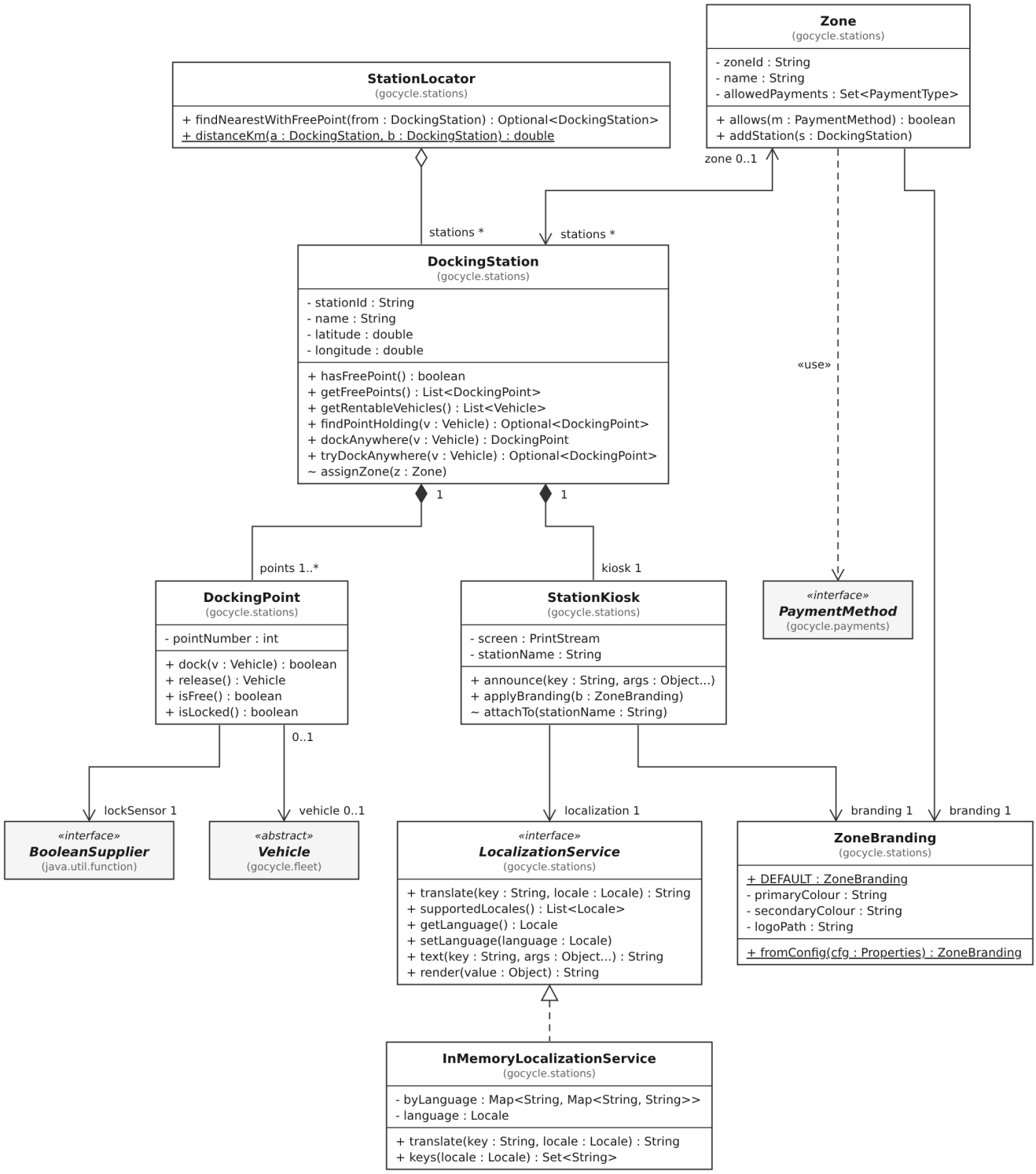
Pricing and payments:



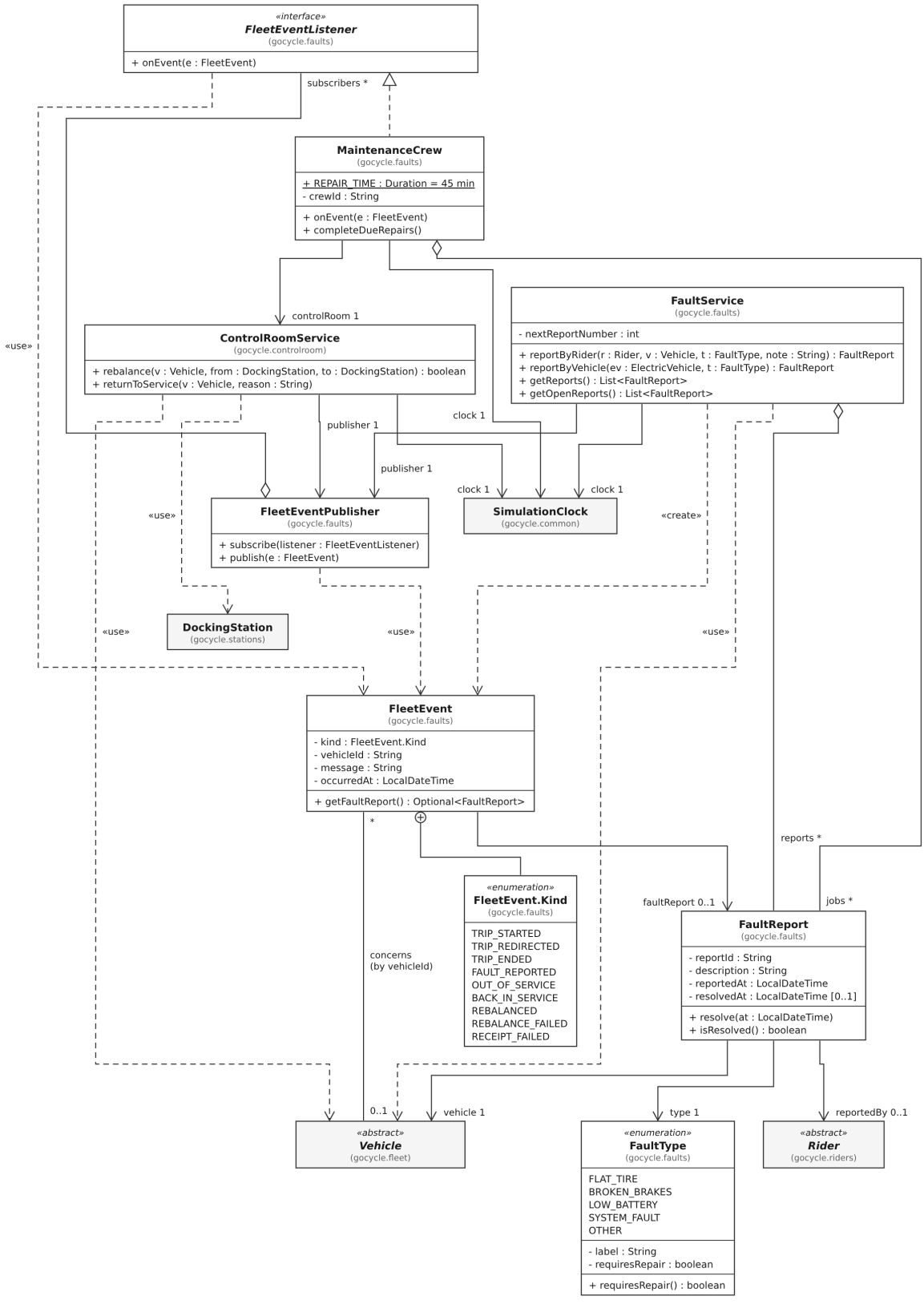
Fleet:


DockingPoint
(gocycle.stations)
0..1
vehicle 0..1
«abstract»
Vehicle
(gocycle.fleet)
- vehicleId : String
- serviceDueOnReturn : boolean
+ getType() : VehicleType
+ checkRentable() : Optional<RentalRefusal>
+ isRentable() : boolean
+ startUse()
+ endUse()
+ markOutOfService()
+ markAvailable()
+ markInTransit()
+ ride(minutes : int)
+ rest(minutes : int)
«abstract»
ElectricVehicle
(gocycle.fleet)
+ MINUTES_PER_PERCENT_CHARGED : int = 5
- lowBatteryReported : boolean
- minutesCharging : int
drainPerMinute() : int
+ checkRentable() : Optional<RentalRefusal>
+ ride(minutes : int)
+ rest(minutes : int)
+ reportSelfFault(type : FaultType)
VehicleFactory
(gocycle.fleet)
+ create(type : VehicleType, id : String) : Vehicle
+ create(type : VehicleType, id : String, charge : int) : Vehicle
«create»
«use»
1
«use»
«create»
«create»
«create»
«use»
battery 1
faultService 1
faultService 1
status 1
«record»
RentalRefusal
(gocycle.fleet)
+ messageKey : String
+ args : Object[]
Battery
(gocycle.fleet)
+ MIN_RENT_PERCENT : int = 20
- chargePercent : int
+ meetsRentThreshold() : boolean
+ drain(percent : int)
+ charge(percent : int)
«enumeration»
FaultType
(gocycle.faults)
ElectricScooter
(gocycle.fleet)
drainPerMinute() : int
+ getType() : VehicleType
ElectricBike
(gocycle.fleet)
drainPerMinute() : int
+ getType() : VehicleType
FaultService
(gocycle.faults)
PedalBike
(gocycle.fleet)
+ getType() : VehicleType
«enumeration»
VehicleType
(gocycle.fleet)
PEDAL_BIKE
E_BIKE
E_SCOOTER
«enumeration»
VehicleStatus
(gocycle.fleet)
AVAILABLE
IN_USE
OUT_OF_SERVICE
IN_TRANSIT

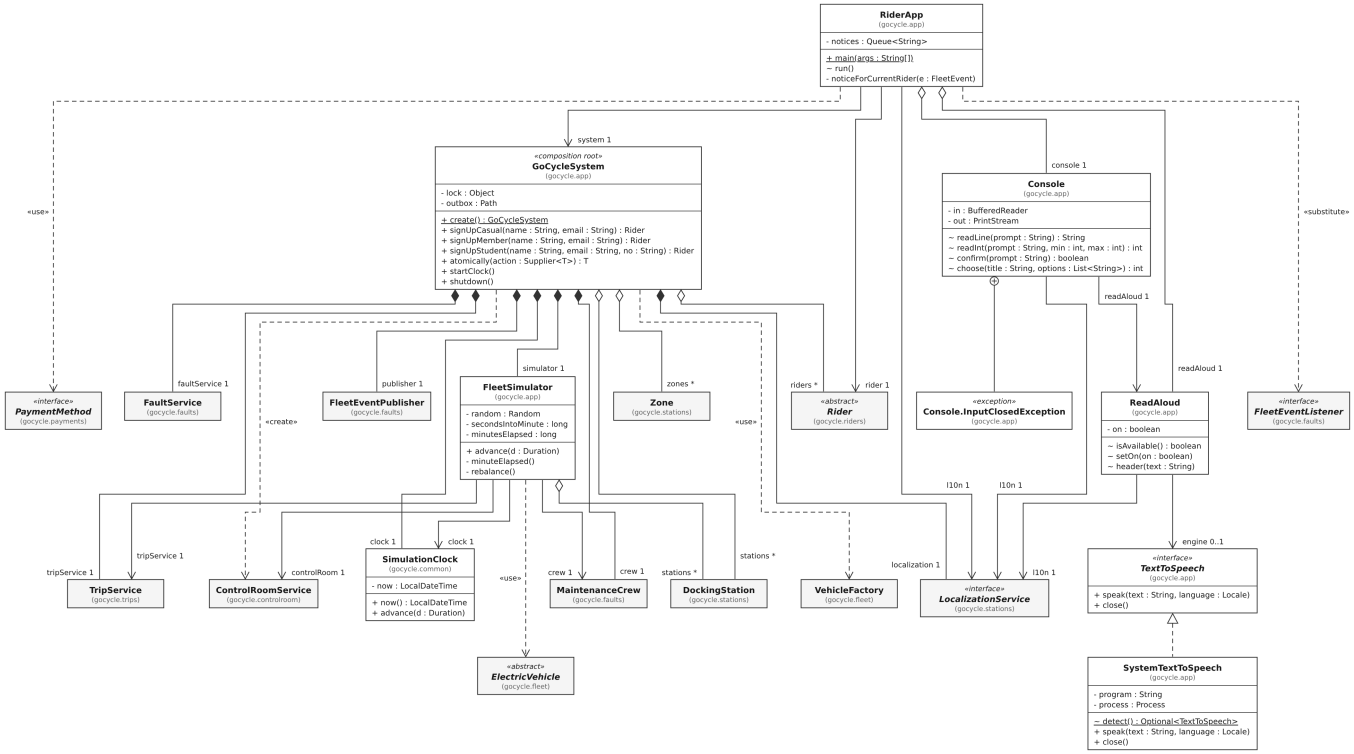
Stations:


Zone
(gocycle.stations)
- zoneId : String
- name : String
- allowedPayments : Set<PaymentType>
+ allows(m : PaymentMethod) : boolean
+ addStation(s : DockingStation)
StationLocator
(gocycle.stations)
+ findNearestWithFreePoint(from : DockingStation) : Optional<DockingStation>
+ distanceKm(a : DockingStation, b : DockingStation) : double
zone 0..1
stations *
stations *
DockingStation
(gocycle.stations)
- stationId : String
- name : String
- latitude : double
- longitude : double
+ hasFreePoint() : boolean
+ getFreePoints() : List<DockingPoint>
+ getRentableVehicles() : List<Vehicle>
+ findPointHolding(v : Vehicle) : Optional<DockingPoint>
+ dockAnywhere(v : Vehicle) : DockingPoint
+ tryDockAnywhere(v : Vehicle) : Optional<DockingPoint>
~ assignZone(z : Zone)
«use»
1
1
points 1..*
kiosk 1
DockingPoint
(gocycle.stations)
- pointNumber : int
+ dock(v : Vehicle) : boolean
+ release() : Vehicle
+ isFree() : boolean
+ isLocked() : boolean
StationKiosk
(gocycle.stations)
- screen : PrintStream
- stationName : String
+ announce(key : String, args : Object...)
+ applyBranding(b : ZoneBranding)
~ attachTo(stationName : String)
«interface»
PaymentMethod
(gocycle.payments)
0..1
lockSensor 1
vehicle 0..1
localization 1
branding 1
branding 1
«interface»
BooleanSupplier
(java.util.function)
«abstract»
Vehicle
(gocycle.fleet)
«interface»
LocalizationService
(gocycle.stations)
+ translate(key : String, locale : Locale) : String
+ supportedLocales() : List<Locale>
+ getLanguage() : Locale
+ setLanguage(language : Locale)
+ text(key : String, args : Object...) : String
+ render(value : Object) : String
ZoneBranding
(gocycle.stations)
+ DEFAULT : ZoneBranding
- primaryColour : String
- secondaryColour : String
- logoPath : String
+ fromConfig(cfg : Properties) : ZoneBranding
InMemoryLocalizationService
(gocycle.stations)
- byLanguage : Map<String, Map<String, String>>
- language : Locale
+ translate(key : String, locale : Locale) : String
+ keys(locale : Locale) : Set<String>

Faults, events and control room:

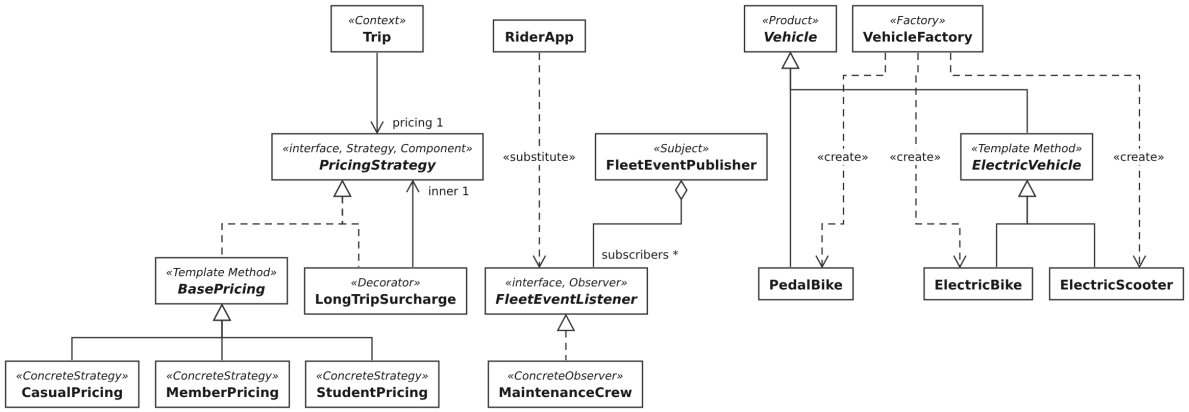


Application layer:



4. Design rationale and patterns

Flexibility: every likely change sits behind a small interface (`PricingStrategy`, `PaymentMethod`, `ReceiptSender`, `FleetEventListener`, `LocalizationService`, `TextToSpeech`), and only `GoCycleSystem` picks the concrete classes. A new fare, payment kind, receipt channel or listener is one new class plus one line of wiring, and `TripService` never changes (QR-10). **Maintainability:** each package owns one part of the domain, shared rules live in one place (`Vehicle.checkRentable()`, `BasePricing.calculate()`, `Money`), and the clock, lock sensor, randomness and speech engine are injected, so tests are deterministic (QR-12).



- **Strategy** (`PricingStrategy`, context `Trip`): rider kinds pay different rates, fixed per trip, so each `Rider` returns a strategy that `TripService` stores on the `Trip`. Benefit: a new fare is one class. Trade-off: riders know concrete strategies.
- **Decorator** (`LongTripSurcharge`): the $5.00 surcharge over 120 min applies to every fare, so one wrapper adds it to any strategy. Benefit: rules stack without edits. Trade-off: each rider must wrap; order matters.
- **Observer** (`FleetEventPublisher`, `MaintenanceCrew`, `RiderApp`): events must reach listeners the services don't know, so one subject notifies many. Benefit: new listeners need no publisher change. Trade-off: a slow listener stalls the synchronous publish.
- **Factory** (`VehicleFactory`): three vehicle kinds, and electric ones need `FaultService`, so one place maps `VehicleType` to a subclass. Benefit: callers see only `Vehicle`. Trade-off: the switch changes per new kind.
- **Template Method** (`ElectricVehicle`, `BasePricing`): `ride()` and `rest()` fix how a battery drains and charges, and `ElectricBike` and `ElectricScooter` fill in only the `drainPerMinute()` hook; `BasePricing` likewise fixes the fare in a `final calculate()` that subclasses give rates to. Benefit: invariants written once. Trade-off: base-class coupling.

SOLID:

S: `Battery` only tracks charge, `DockingPoint` only locks and unlocks, `StationLocator` only finds the nearest free station, and `TripService` only runs the trip lifecycle, so each has one reason to change.

O: `TripService`, `Trip` and `FleetEventPublisher` are closed to edits but open to new `PricingStrategy`, `PaymentMethod`, `ReceiptSender` and `FleetEventListener` classes; `LongTripSurcharge` adds a rule by wrapping, not editing.

L: any `Rider`, `Vehicle`, `PricingStrategy` or `PaymentMethod` subtype works wherever its parent is expected; `ElectricVehicle.checkRentable()` calls `super` and only adds the battery refusal, so it never weakens the parent's rule.

I: role interfaces stay small: `FleetEventListener` has one method, and `ReceiptSender`, `PricingStrategy` and `TextToSpeech` have two each, so no client depends on methods it does not use.

D: `TripService` depends on `ReceiptSender`, `StationKiosk` on `LocalizationService`, `DockingPoint` on a `BooleanSupplier` lock sensor and `ReadAloud` on `TextToSpeech`; `GoCycleSystem` injects the real classes and tests inject fakes. **Limits:** the factory switch and `double` money.